UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Sistemas Operativos

Cambio de Contexto y Multitarea

Práctica 3

Presenta:
Rojo Peña Manuel Ianluck

Profesor:
Javier León Cotonieto

Ayudante laboratorio:
Adrián Martínez Manzo

Grupo: 7004, 2026-2

Fecha de entrega:
9 de marzo, 2026

Introducción

Una vez implementada la diferenciación entre Modo Kernel y Modo Usuario para el manejo del hardware, contamos con la base funcional de los sistemas operativos modernos. El siguiente paso es agregar otra capacidad fundamental: la programación multitarea con cambios de contexto.

Para lograrlo será necesario comprender a fondo la *Gestión de Memoria y Pilas*, específicamente el funcionamiento del **Process Stack Pointer** (PSP) y la estructura del **Stack Frame**. Igualmente, deberemos analizar el *Manejo de Excepciones del Sistema*, en particular el rol de la excepción `PendSV`, que permite diferir los cambios de contexto de forma segura sin bloquear el sistema.

Planteamiento del Problema

Debemos diseñar e implementar un *Conmutador de Contexto* construido en ensamblador y C, capaz de alternar la ejecución entre múltiples tareas guardando y restaurando su estado interno sin pérdida de datos. El sistema debe dar la ilusión de ejecutar varios procesos simultáneamente, retomando cada tarea exactamente en el punto donde fue suspendida.

En esta práctica se evoluciona el Kernel existente para sostener la ejecución de tres procesos independientes, que se alternan mediante el botón físico conectado a interrupciones de hardware y que se verán reflejadas a través de cinco LEDs conectado a la placa.

Desarrollo

Mecanismo Cambio de Contexto

El flujo que ocurre al presionar el botón es el siguiente:

1. **Interrupción Física:** El usuario presiona el botón, disparando la rutina `IO_IRQ_BANK0_Handler`. Esta limpia el flag de interrupción y escribe el bit `PENDSVSET` en `SCB_ICSR`.

2. **Solicitud de PendSV:** Para no bloquear el sistema con trabajo pesado dentro de un `IRQ` de hardware, el handler delega el cambio de contexto a la excepción `PendSV`. Esta tiene la prioridad más baja, así que se dispara al salir del `IRQ` del botón.

3. **Guardado (Save):** `PendSV_Handler` deshabilita interrupciones (`cpsid i`), lee el **PSP** con `mrs r0, psp` y guarda manualmente los registros $r4$ - $r11$ en la pila de la tarea actual (el Software Frame). El procesador ya guardó $r0$ - $r3$, $r12$, lr, pc y $xPSR$ automáticamente (el Hardware Frame).

4. **Cambio de ID:** Se lee `current_task_id`, se actualiza el apuntador de pila de la tarea actual (`sp_taskN` con `N` $\in \{1, 2, 3\}$) y se calcula el ID de la próxima tarea en *Round - Robin*: $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$.

5. **Restauración (Load):** Se carga el **PSP** guardado de la nueva tarea, se restauran $r4$ - $r11$ y se actualiza el **PSP**. Al retornar con `bx 0xFFFFFFFD` el procesador desapila automáticamente el *Hardware Frame* y la nueva tarea retoma su ejecución.

Diagrama del Stack Frame

En las siguientes tablas se muestra la estructura completa de la pila de cada tarea durante un cambio de contexto.

Software Frame. Guardado manualmente en `pendsv.s`

PSP Offset	Registro	Descripción
PSP + 0x00	r4	Guardado con: `subs r0,#32 + stmia r0!,{r4-r7}`
PSP + 0x04	r5	`stmia r0!,{r4-r7}`
PSP + 0x08	r6	`stmia r0!,{r4-r7}`
PSP + 0x0C	r7	`stmia r0!,{r4-r7}`
PSP + 0x10	r8	`mov r4,r8 + stmia r0!,{r4-r7}`
PSP + 0x14	r9	`mov r5,r9 + stmia r0!,{r4-r7}`
PSP + 0x18	r10	`mov r6,r10 + stmia r0!,{r4-r7}`
PSP + 0x1C	r11	`mov r7,r11 + stmia r0!,{r4-r7}`

Hardware Frame. Guardado automáticamente por el procesador al entrar la excepción

PSP Offset	Registro	Descripción
`PSP + 0x20`	`r0`	Guardado automáticamente al detectar la excepción
`PSP + 0x24`	`r1`	Guardado automáticamente
`PSP + 0x28`	`r2`	Guardado automáticamente
`PSP + 0x2C`	`r3`	Guardado automáticamente
`PSP + 0x30`	`r12`	Guardado automáticamente
`PSP + 0x34`	`lr`	Dirección de retorno de la tarea
`PSP + 0x38`	`pc`	Siguiente instrucción a ejecutar
`PSP + 0x3C`	`xpsr`	Flags de estado + Thumb bit

Cabe mencionar que para $xPSR$, las flags de estado, son bits dentro de este registro que el procesador actualiza automáticamente después de operaciones aritméticas/lógicas. Los usa para tomar decisiones en saltos condicionales.

Mientras que **Thumb bit**, `Cortex-M0+` solo ejecuta estas instrucciones **Thumb** (*16/32* bits), nunca **ARM** clásico (32 bits). El bit T en `xPSR` indica al procesador en qué modo está:

- T = 1: modo **Thumb** → normal, todo funciona
- T = 0: modo **ARM** → en `M0+` causa un *HardFault* inmediato

Por eso en `task_init_stack` inicializamos $xPSR$ con `0x01000000`, este es exactamente el bit 24 (T) en 1, si pusiera `0x00000000` el procesador intentaría ejecutar modo **ARM** al iniciar la tarea y se rompería instantáneamente.

Tabla de Flujo Round-Robin

En el scheduler implementamos un *Round-Robin* cooperativo activado por botón. La siguiente tabla resume las tres transiciones posibles:

Tarea activa	Botón	Guarda en	next ID	Carga de	Resultado
Tarea 0 (`ID` = 0)	Presionado	`sp_task1`	`ID` → 1	`sp_task2`	**Tarea 1**
Tarea 1 (`ID` = 1)	Presionado	`sp_task2`	`ID` → 2	`sp_task3`	**Tarea 2**
Tarea 2 (`ID` = 2)	Presionado	`sp_task3`	`ID` → 0	`sp_task1`	**Tarea 0**

Implementación Tercera Tarea

Para soportar tres tareas simultáneas se modificaron tres archivos `main.c`, `pendsv.s` y `user_app.c`. Cabe mencionar que, la implementación interesante está en el archivo `pendsv.s`, puesto que en `main.c` solo agreagamos la instrucción de la nueva tarea y la inicializamos; mientras que en `user_app.c` definimos el comportamiento de esa tercer tarea:

`main.c` Declaración de memoria y apuntador

Se agregaron la pila y el apuntador de la tercera tarea (`task_lullaby`), con alineación obligatoria a 8 bytes según la **AAPCS** (*ARM Application Binary Interface*):

```c
/* 8-byte aligned stacks (ARM AAPCS requirement) */
uint32_t task1_stack[STACK_SIZE] __attribute__((aligned(8)));
uint32_t task2_stack[STACK_SIZE] __attribute__((aligned(8)));
uint32_t task3_stack[STACK_SIZE] __attribute__((aligned(8)));

uint32_t *sp_task1;
uint32_t *sp_task2;
uint32_t *sp_task3;

volatile uint32_t current_task_id = 0;

/* External Declarations */
extern void wrapper_svc(void);
extern void task_counter(void);
extern void task_animation(void);
extern void task_lullaby(void);

/**
 * @brief Kernel entry point.
 * @return Never returns.
 */
int main(void) {
    install_vector_table();
    exception_set_exclusive_handler((enum exception_number)11,
   wrapper_svc);

    // Set PendSV priority to lowest (0xFF)
    // Bits 23:16 control PRI_14 (PendSV)
    SHPR3 |= (0xFF << 16);

    // Hardware init
    k_gpio_init(LED_DEBUG, 1);

    k_gpio_init(BTN_PIN, 0);
    k_gpio_pullup(BTN_PIN);
    k_gpio_irq_enable(BTN_PIN, 0);

    NVIC_ISER = (1 << 13);

```

```c
    // Initialize tasks
    task_init_stack(task1_stack, task_counter, &sp_task1);
    task_init_stack(task2_stack, task_animation, &sp_task2);
    task_init_stack(task3_stack, task_lullaby, &sp_task3); //added the task3 init

    os_start_first_task();

    while (1);
}
```

Código 1: Task3 en `main.c`

La tabla de vectores es una lista de direcciones de memoria, cuando ocurre un evento (como presionar el botón), el hardware busca en esta tabla a qué función debe saltar. En `install_vector_table`, es como si le dijéramos al procesador: "*Cuando ocurra la interrupción 13 (Botón), ve a `IO_IRQ_BANK0_Handler` y cuando ocurra la excepción 14 (PendSV), ve a `PendSV_Handler`*". Luego, le decimos al hardware dónde está esta nueva tabla escribiendo en el registro `SCB_VTOR`.

Con la instrucción `SHPR3 |= (0xFF « 16)` le damos a **PendSV** la prioridad más baja posible, con esto aseguramos que el cambio de contexto nunca interrumpa procesos de hardware urgentes.

`user_app.c` **Tercera tarea (*task_lullaby*)**

Se implementó task_lullaby, que enciende secuencialmente los cinco LEDs de acuerdo a la melodía de una canción. Los tiempos se calcularon con la constante `41,700,000 ciclos/segundo` correspondiente al **RP2040** a 125 MHz usando la función auxiliar dada por el ayudante `delay_cycles_exact`:

```c
/**
 * @brief Melody of a lullaby using LEDs
 */
void task_lullaby(void) {
    setup_leds();

    // Persistent state variables on the stack
    int pins[] = {LED_0, LED_1, LED_2, LED_3, LED_4};

    while (1) {
      // Turn off all
      for (int i = 0; i < 5; i++)
    sys_gpio_set(pins[i], 0);

      //Countdown
      sys_gpio_set(pins[4], 1);
      sys_gpio_set(pins[0], 1);
      sys_gpio_set(pins[3], 1);
      sys_gpio_set(pins[1], 1);
```

```
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(41700000); //3
        sys_gpio_set(pins[4], 0);
        sys_gpio_set(pins[0], 0);
        delay_cycles_exact(41700000); //2
        sys_gpio_set(pins[3], 0);
        sys_gpio_set(pins[1], 0);
        delay_cycles_exact(41700000); //1
        sys_gpio_set(pins[2], 0);

        delay_cycles_exact(20850000); //0

        /* Firts part*/
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(9000000);
        sys_gpio_set(pins[2], 0);
        //A
        sys_gpio_set(pins[1], 1);
        delay_cycles_exact(9000000);
        sys_gpio_set(pins[1], 0);
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(9000000);
        sys_gpio_set(pins[2], 0);
        //P-A
        sys_gpio_set(pins[4], 1);
        sys_gpio_set(pins[1], 1);
        delay_cycles_exact(22000000);
        sys_gpio_set(pins[4], 0);
        sys_gpio_set(pins[1], 0);
        //A
        sys_gpio_set(pins[1], 1);
        delay_cycles_exact(14000000);
        sys_gpio_set(pins[1], 0);
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(9000000);
        sys_gpio_set(pins[2], 0);
        //A
        sys_gpio_set(pins[1], 1);
        delay_cycles_exact(9000000);
        sys_gpio_set(pins[1], 0);
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(9000000);
        sys_gpio_set(pins[2], 0);
        //P-A
        sys_gpio_set(pins[4], 1);
        sys_gpio_set(pins[1], 1);
        delay_cycles_exact(31275000);//
        sys_gpio_set(pins[4], 0);
```

```
        sys_gpio_set(pins[1], 0);
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(10000000);
        sys_gpio_set(pins[2], 0);
        //A
        sys_gpio_set(pins[1], 1);
        delay_cycles_exact(25000000);
        sys_gpio_set(pins[1], 0);
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(5000000);
        sys_gpio_set(pins[2], 0);
        delay_cycles_exact(5000000);
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(7500000);
        sys_gpio_set(pins[2], 0);
        //I
        sys_gpio_set(pins[3], 1);
        delay_cycles_exact(6000000);
        sys_gpio_set(pins[3], 0);
        delay_cycles_exact(2000000);
        //I
        sys_gpio_set(pins[3], 1);
        delay_cycles_exact(8500000);
        sys_gpio_set(pins[3], 0);
        //P
        sys_gpio_set(pins[4], 1);
        delay_cycles_exact(5000000);
        sys_gpio_set(pins[4], 0);
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(6000000);
        sys_gpio_set(pins[2], 0);
        //A
        sys_gpio_set(pins[1], 1);
        delay_cycles_exact(5000000);
        sys_gpio_set(pins[1], 0);
        //M
        sys_gpio_set(pins[0], 1);
        delay_cycles_exact(20000000);
        sys_gpio_set(pins[0], 0);
        //
        delay_cycles_exact(9000000);
        //A
        sys_gpio_set(pins[1], 1);
        delay_cycles_exact(20000000);
        sys_gpio_set(pins[1], 0);
        delay_cycles_exact(5000000);
        //A
        sys_gpio_set(pins[1], 1);
```

```
        delay_cycles_exact(34500000);
        sys_gpio_set(pins[1], 0);
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(3500000);
        sys_gpio_set(pins[2], 0);
        delay_cycles_exact(3500000);
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(1000000);
        sys_gpio_set(pins[2], 0);
        //I
        sys_gpio_set(pins[3], 1);
        delay_cycles_exact(4500000);
        sys_gpio_set(pins[3], 0);
        delay_cycles_exact(4500000);
        //I
        sys_gpio_set(pins[3], 1);
        delay_cycles_exact(10000000);
        sys_gpio_set(pins[3], 0);
        //P
        sys_gpio_set(pins[4], 1);
        delay_cycles_exact(5000000);
        sys_gpio_set(pins[4], 0);
        //I
        sys_gpio_set(pins[3], 1);
        delay_cycles_exact(6000000);
        sys_gpio_set(pins[3], 0);
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(6000000);
        sys_gpio_set(pins[2], 0);
        //A
        sys_gpio_set(pins[1], 1);
        delay_cycles_exact(6000000);
        sys_gpio_set(pins[1], 0);
        //M
        sys_gpio_set(pins[0], 1);
        delay_cycles_exact(10000000);
        sys_gpio_set(pins[0], 0);

        /* Second part */
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(20000000);
        sys_gpio_set(pins[2], 0);
        delay_cycles_exact(5000000);
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(9000000);
        sys_gpio_set(pins[2], 0);
        //A
```

```
        sys_gpio_set(pins[1], 1);
        delay_cycles_exact(9000000);
        sys_gpio_set(pins[1], 0);
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(9000000);
        sys_gpio_set(pins[2], 0);
        //I-M
        sys_gpio_set(pins[3], 1);
        sys_gpio_set(pins[0], 1);
        delay_cycles_exact(31275000);
        sys_gpio_set(pins[3], 0);
        sys_gpio_set(pins[0], 0);
        //A
        sys_gpio_set(pins[1], 1);
        delay_cycles_exact(14000000);
        sys_gpio_set(pins[1], 0);
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(9000000);
        sys_gpio_set(pins[2], 0);
        //A
        sys_gpio_set(pins[1], 1);
        delay_cycles_exact(9000000);
        sys_gpio_set(pins[1], 0);
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(9000000);
        sys_gpio_set(pins[2], 0);
        //I-M
        sys_gpio_set(pins[3], 1);
        sys_gpio_set(pins[0], 1);
        delay_cycles_exact(25000000);//
        sys_gpio_set(pins[3], 0);
        sys_gpio_set(pins[0], 0);
        //
        delay_cycles_exact(9000000);
        //P-D
        sys_gpio_set(pins[4], 1);
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(10000000);//
        sys_gpio_set(pins[4], 0);
        sys_gpio_set(pins[2], 0);
        //I-A
        sys_gpio_set(pins[3], 1);
        sys_gpio_set(pins[1], 1);
        delay_cycles_exact(25000000);//
        sys_gpio_set(pins[3], 0);
        sys_gpio_set(pins[1], 0);
        //
        delay_cycles_exact(9000000);
        //D
```

```c
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(5500000);
        sys_gpio_set(pins[2], 0);
        delay_cycles_exact(5500000);
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(8000000);
        sys_gpio_set(pins[2], 0);
        //I
        sys_gpio_set(pins[3], 1);
        delay_cycles_exact(7000000);
        sys_gpio_set(pins[3], 0);
        //P
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(7000000);
        sys_gpio_set(pins[2], 0);
        //I
        sys_gpio_set(pins[3], 1);
        delay_cycles_exact(5000000);
        //D
        sys_gpio_set(pins[2], 1);
        delay_cycles_exact(5000000);
        //M
        sys_gpio_set(pins[0], 1);
        delay_cycles_exact(58000000);
        sys_gpio_set(pins[3], 0);
        sys_gpio_set(pins[2], 0);
        sys_gpio_set(pins[0], 0);

        //
        delay_cycles_exact(90000);

        /* Again */
        //It repeats again

        delay_cycles_exact(41700000);
    }
}
```

Código 2: Algoritmo para `task_lullaby` en `user_app.c`

Se puede apreciar mejor el funcionamiento de esta tarea en el video de evidencias.

`pendsv.s` Conmutador con Round-Robin de 3 tareas

Se amplió la lógica de saltos condicionales para distinguir tres IDs. El error más frecuente al escalar de 2 a 3 tareas es olvidar el segundo CMP en el bloque de carga, haciendo que la tarea 1 cargue el contexto de la tarea 3 en lugar de la 2:

```asm
.syntax unified
.cpu cortex-m0plus
.thumb
```

```

.global PendSV_Handler
.type PendSV_Handler, %function

/* Context Switching Routine for PendSV Exception */
PendSV_Handler:
    cpsid i       /* Disable interrupts to ensure atomicity during context switch */
    mrs r0, psp   /* Mov PSP (Process Stack Pointer) to R0 for manipulation */

    /* Hardware automatically saves R0-R3, R12, LR, PC, and xPSR on the stack when an exception occurs.  We need to save the remaining registers (R4-R11) manually. */

    subs r0, #32       /* For 8 registers (R4-R11), we need 32 bytes (8 registers * 4 bytes each) */
    stmia r0!, {r4-r7} /* Store R4-R7 in the lower part of the stack frame */

    /* Move R8-R11 to R4-R7 temporarily to store them */
    mov r4, r8
    mov r5, r9
    mov r6, r10
    mov r7, r11

    stmia r0!, {r4-r7} /* Store R8-R11 in the upper part of the stack frame (offset +16) */
    subs r0, #32 /* Restore R0 to point to the start of the full stack frame (including hardware-saved registers) */
    ldr r2, =current_task_id /* Read current_task_id to determine where to save the current context */
    ldr r3, [r2]            /* r3 now holds the current task ID */

    /* If current_task_id is 0, we just switched from task 0 to task 1, so we save the context of task 0 in sp_task1. If it's 1, we save in sp_task2. */
    cmp r3, #0
    beq save_task1   /* Task 0 is running, then we save to sp_task1 */
    cmp r3, #1       /* Is task 1 is running? */
    beq save_task2   /* Yes: save to sp_task2 */
    b save_task3     /* Otherwise: task 2 is running and save to sp_task3 */

save_task3:
    /* Save context of task 2 and prepare task 0 as next (Round-Robin wrap) */
    ldr r1, =sp_task3
    str r0, [r1]            /* Update sp_task3 with the current stack pointer */
    movs r3, #0             /* Prepare the ID for the next task (0) */
    b switch_context        /* Jump to the context switch part */
```

```

save_task2:
    ldr r1, =sp_task2
    str r0, [r1]            /* Update sp_task2 with the current stack pointer of task 2 */
    movs r3, #2             /* Prepare the ID for the next task (2) MINE */
    b switch_context        /* Jump to the context switch part */

save_task1:
    ldr r1, =sp_task1
    str r0, [r1]            /* Update sp_task1 with the current stack pointer of task 1 */
    movs r3, #1             /* Prepare the ID for the next task (1) */

switch_context:
    str r3, [r2]            /* Store the new current_task_id (the task we are switching to) */

    /* If current_task_id was 0, we need to load task 1's context. If it was 1, we load task 2's context. */
    cmp r3, #0
    beq load_task1   /* New task is 0, then load sp_task1 */
    cmp r3, #1       /* Is new task is 1? */
    beq load_task2   /* Yes: load sp_task2 */
    b   load_task3   /* Otherwise: new task is 2 and load sp_task3 */

load_task3:
    /* Load stack pointer of Task 2 */
    ldr r1, =sp_task3
    ldr r0, [r1]            /* store in r0 the sp_task3 value, which is the stack pointer of task 3 */
    b restore_context

load_task2:
    ldr r1, =sp_task2
    ldr r0, [r1]            /* store in r0 the sp_task2 value, which is the stack pointer of task 2 */
    b restore_context

load_task1:
    ldr r1, =sp_task1
    ldr r0, [r1]            /* store in r0 the sp_task1 value, which is the stack pointer of task 1 */

/* R0 points to the stack frame of the task we are switching to. We need to restore R4-R11 from this stack frame. */
restore_context:
    /* Recover R8-R11 registers first (they are in the upper part of the stack frame) */
    adds r0, #16
    ldmia r0!, {r4-r7}
```

```
    mov r8, r4
    mov r9, r5
    mov r10, r6
    mov r11, r7

    /* Recover R4-R7 registers (they are in the lower part of the stack frame) */
    subs r0, #32            /* Get back to the start of the stack frame */
    ldmia r0!, {r4-r7}      /* Restore R4-R7 from the stack frame */

    /* At this point, R0-R3, R12, LR, PC, and xPSR are still on the stack and will be automatically restored when we exit the exception handler. We just need to update the PSP to point to the new stack frame for the task we are switching to. */
    adds r0, #16
    msr psp, r0 /* Update PSP to point to the new stack frame */
    cpsie i     /* Enable interrupts again before exiting the exception handler */

    /* Special return from exception to switch to the new task. The value 0xFFFFFFFD tells the processor to return to Thread mode and use the Process Stack Pointer (PSP) for the new task. */
    ldr r0, =0xFFFFFFFD
    bx r0

.align 4
```

Código 3: `pendsv.s`

Para que una tarea se pueda reanudarse, primero tiene que existir, como las tareas nunca se han ejecutado, tenemos que falsificar su estado inicial.
la función `task_init_stack` toma un bloque de memoria y coloca los registros exactamente en el orden en que el hardware los esperaría si la tarea hubiera sido pausada. Se coloca el bit **Thumb** en $xPSR$ (`0x01000000`), el pc apuntando al inicio de `task_counter`, y el lr con el valor `0xFFFFFFFD`. Deja 8 espacios vacíos para los registros de software ($r4$ - $r11$) y guarda el topde de la pila en la variable `sp_taskN`.

`os_start_first_task` es el *motor de arranque*, carga la pila falsa de `sp_task1`. Configura el procesador para que deje de usar la pila principal del sistema y empiece a usar la pila de procesos con `msr control, r0`. Extrae manualmente los registros iniciales y, con `bx r6`, salta a la primera instrucción de la tarea 1.

El Disparador `IO_IRQ_BANK0_Handler`, mientras la tarea 1 corre, si se presiona el botón el hardware salta a esta función. En lugar de cambiar de tarea aquí mismo, esta función simplemente levanta una bandera: `SCB_ICSR |= ICSR_PENDSVSET`, esto solicita una excepción **PendSV**. Como configuramos **PendSV** con la prioridad más baja, el procesador termina de atender el botón, verifica si hay otras urgencias, y como no las

hay, dispara el `PendSV_Handler`.

En `PendSV_Handler` ocurre el cambio de contexto. Se divide en 4 pasos lógicos:

1. Extraer y Guardar (Save):

```
    cpsid i          // Apaga interrupciones para que nadie nos moleste
    mrs r0, psp      // Copia el apuntador de la pila actual (PSP) a R0
    subs r0, #32     // Hace espacio para 8 registros (R4-R11) = 32 bytes

```

2. Enrutar y Guardar el Puntero (Switch)

```
        ldr r2, =current_task_id
        ldr r3, [r2]
        cmp r3, #0
        beq save_task1
        ...

    save_task1:
        ldr r1, =sp_task1
        str r0, [r1]     /* Guarda el apuntador actualizado en la variable sp_task1 */
        movs r3, #1      /* Elige que la siguiente tarea sea la 1 */

```

   Aquí revisamos el `ID` de la tarea que acaba de ser interrumpida. Guardamos su apuntador de pila $r0$ en su `sp_taskN` para no perderlo. Inmediatamente después, decidimos cuál es el `ID` de la siguiente tarea a ejecutar y lo guardamos en `current_task_id`.

3. Cargar el Nuevo Apuntador (Load)

```
      cmp r3, #0
      beq load_task1
      ...

  load_task2:
      ldr r1, =sp_task2
      ldr r0, [r1]     /* Lee el puntero de pila de la Tarea 2 hacia r0 */

```

   Ahora que sabemos que tarea toca, vamos a su variable `sp_taskN`, tomamos su apuntador de pila donde se quedó la última vez y lo ponemos en $r0$.

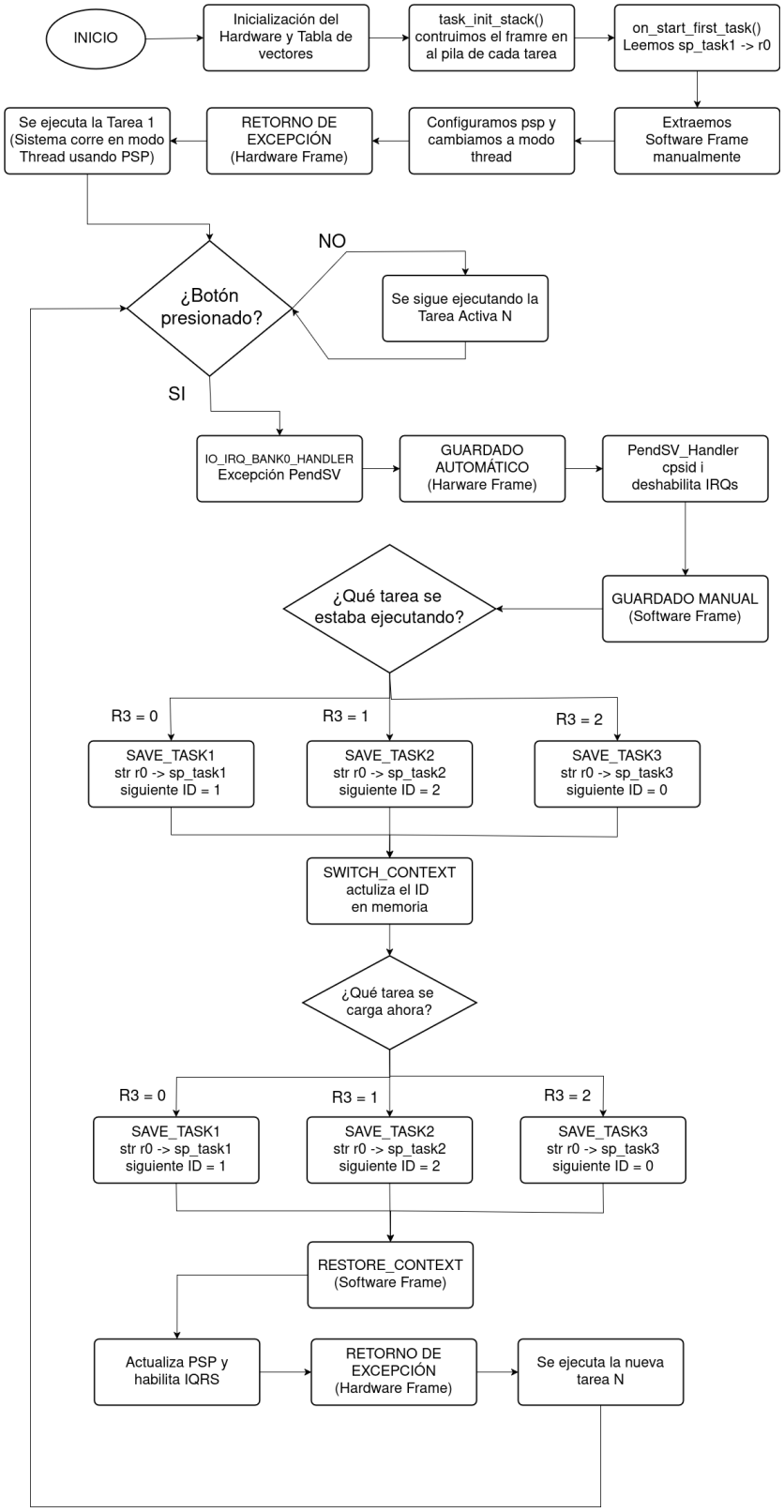
4. Restaurar y Arrancar (Restore)

```
    adds r0, #16
    msr psp, r0      /* Le decimos al procesador: "Este es el
   nuevo stack pointer" */
    cpsie i          /* Volvemos a encender las interrupciones */
    ldr r0, =0xFFFFFFFD
    bx r0            /* Retorno */

```

Recuperamos los registros $r4$ - $r11$ de la nueva tarea. Luego, actualizamos físicamente el registro del hardware y finalmente, la instrucción `bx 0xFFFFFFFD` le dice a ARM que terminó la excepción, regresa a modo Thread, usa el *psp*, y saca automáticamente $r0$ - $r3$, $r12$, lr, pc y $xPSR$ de la nueva pila. Y así es como pc salta donde se había quedado la nueva tarea.

Diagrama de flujo del programa



Problema de Alineación a 8 Bytes

La **ARM AAPCS** exige que el *sp* esté alineado a 8 bytes en el momento de llamar funciones (`push`/`pop`, `ldmia`, `stmia`). Si la pila no está alineada, ocurren los siguientes problemas concretos:

Problema	**Consecuencia**
UsageFault / HardFault	El procesador detecta acceso desalineado en operaciones `LDMIA`/`STMIA` de 8 bytes y lanza una excepción de fallo en tiempo de ejecución.
Corrupción del Hardware Frame	Al entrar `PendSV`, el procesador empuja el *Hardware Frame* (8 registros = 32 bytes) con padding opcional. Si *sp* no es múltiplo de 8, el frame queda en una dirección errónea y los valores restaurados son basura.
Fallo silencioso en STMIA/LDMIA	Las instrucciones de carga/almacenamiento múltiple pueden leer o escribir en direcciones incorrectas sin señal de error visible, corrompiendo datos de otras tareas o del kernel.
Comportamiento no determinista	El programa puede funcionar intermitentemente según el contenido de la memoria adyacente, haciendo el bug difícil de reproducir y diagnosticar.

El atributo `__attribute__((aligned(8)))` en la declaración de cada pila garantiza que el compilador ubique el buffer en una dirección múltiplo de 8, cumpliendo el requisito del ABI y evitando todos los problemas anteriores.

Evidencias

Evidencia Práctica 3 Video

Concluciones

Esta práctica demostramos en la implementación que el cambio de contexto es el *corazón* de cualquier sistema multitarea:

- La división ***Hardware Frame*/*Software Frame*** no es arbitraria: el procesador **ARM** `Cortex-M0+` gestiona automáticamente el primero, pero el código del sistema operativo es responsable explícito del segundo.
- **PendSV** es la excepción correcta para el cambio de contexto precisamente porque tiene la prioridad más baja, garantizando que el switch ocurra solo cuando no hay otro trabajo pendiente.
- Escalar de 2 a 3 tareas en ensamblador requiere revisar cada punto de decisión binaria (`cmp`/`beq`) para convertirlo en un árbol de tres ramas, tanto en el bloque `save` como en el bloque `load`.
- La alineación a 8 bytes no es opcional: una pila desalineada puede producir fallos de hardware silenciosos o HardFaults difíciles de diagnosticar.
- El valor especial `0xFFFFFFFD` como `EXC_RETURN` comunica al procesador que debe regresar a Thread mode usando el *psp*, completando así el ciclo de restauración sin instrucciones adicionales.